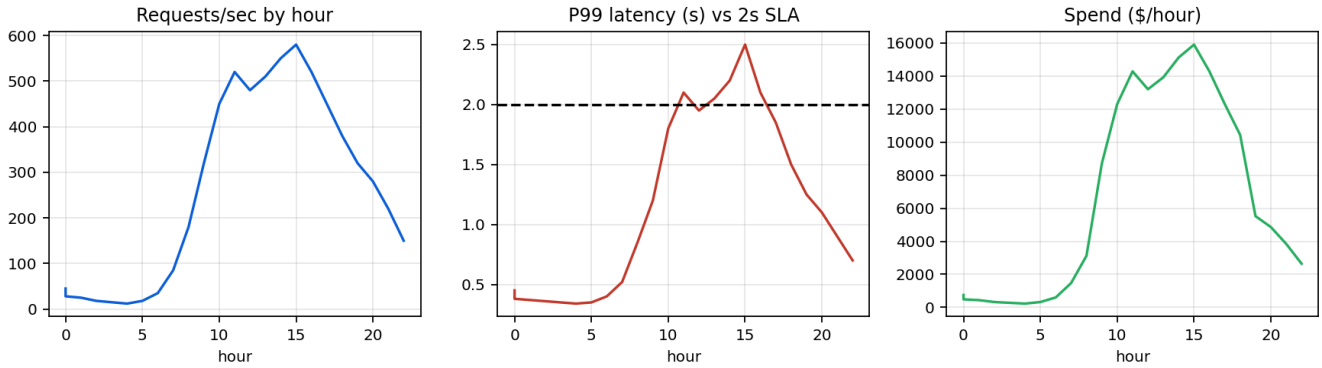
GlobalAI LLM Platform: Monitoring, Auto-Scaling and Cost Optimization

Module 3 Hands-On Practice report (SRE role). All configuration files are in the accompanying package.

Important about the evidence in this report. No live GPU cluster, Prometheus or Grafana instance was available when this package was produced. Configuration files were syntax-validated (YAML/JSON/XML parse) and the routing module was executed against the 26 provided test prompts. The load-test and cost numbers below come from **models** (analysis/hpa_simulation.py, analysis/traffic_and_cost_analysis.py) driven by the provided traffic CSV, not from measurements on a running system. Grafana screenshots and the real JMeter run still need to be captured on your own stack; Section 2 and 5 say exactly what to capture.

Starting point

Scenario: 50M requests/day, 500 clients, response times above 30 s versus a 2 s SLA, $3M/month at 30% utilization, and a $10M Black Friday outage. The provided traffic data shows a peak of 580 req/s against a 12 req/s overnight trough (48.3x swing), P99 above the 2 s SLA in 5 of 24 hourly samples, and an incident window reaching 1500 req/s with P99 of 18 s, 18% errors and the cache hit ratio collapsing to 4%. The incident peak is 3.3x the normal 10:00 load.



Requirement 1: Prometheus monitoring configuration

Files: `monitoring/prometheus.yml`, `recording_rules.yml`, `alert_rules.yml`. Scrape interval is 15 s. Latency is a histogram (`llm_request_duration_seconds`, the request_latency_seconds metric) so P50/P95/P99 can be computed. Other custom metrics: `llm_tokens_processed_total`, `llm_gpu_utilization` (plus DCGM), `llm_cost_dollars_total`, `llm_cost_per_request_dollars`, and `llm_request_queue_depth`, the signal the HPA scales on. Service discovery uses `kubernetes_sd_configs` (role pod, namespace llm-production, keep app=llm-inference with scrape annotation), so HPA-created pods are scraped automatically instead of relying on static targets.

```
# GlobalAI LLM Platform - Prometheus configuration (Activity 1, Step 1)
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  scrape_timeout: 10s
  external_labels:
    environment: production
    service: globalai-llm

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']
      api_version: v2

rule_files:
```

Alert	Condition	For	Route
LLMHighLatencyP95	P95 > 1.5 s	2m	Slack platform
LLMSLABreachP99	P99 > 2 s (SLA)	1m	PagerDuty
LLMHighErrorRate	5xx ratio > 1%	2m	PagerDuty + Slack
LLMServiceDown	up == 0	1m	PagerDuty
LLMCostAnomaly	Hourly spend > 1.2 x 7-day baseline	15m	Slack finance
HighRequestQueueDepth	avg queue per pod > 30	2m	Slack platform

Requirement 2: Grafana dashboard

File: `monitoring/grafana_dashboard.json` (import via Dashboards > Import; choose the Prometheus data source). 21 panels in rows for the four dimensions plus a scaling row.

Dimension	Panels	Why this metric / threshold
Performance	Throughput req/s; P50/P95/P99 latency	Percentiles, not averages, because LLM latency is long-tailed. Threshold line at the 2 s SLA; alert at P95 1.5 s gives lead time.
Availability	Uptime % (gauge, green at 99.95), error rate, healthy pods, health status	99.95% allows about 21.6 minutes of downtime per 30 days. Error-rate alert at 1% per the activity; yellow from 0.1%.
Usage	Active users, requests by endpoint, token consumption	Tokens drive cost and GPU load; endpoint split shows which workload to cache or route.
Cost	Hourly spend vs baseline+20%, cost per request, projected monthly, cache hit ratio	Cost anomaly threshold is 20% above the 7-day baseline; cache hit target 25-35% (red below 25).

Alert notification configuration is in `monitoring/alertmanager.yml`: PagerDuty for anything labelled pagerduty="true", Slack #llm-platform-alerts for the platform team, #llm-cost-alerts for finance, with an inhibit rule so a P99 breach suppresses the P95 warning. Webhook URLs and the PagerDuty key are placeholders.

To capture for submission: one screenshot per row of the dashboard while JMeter runs, and a screenshot of the Alertmanager receivers page.

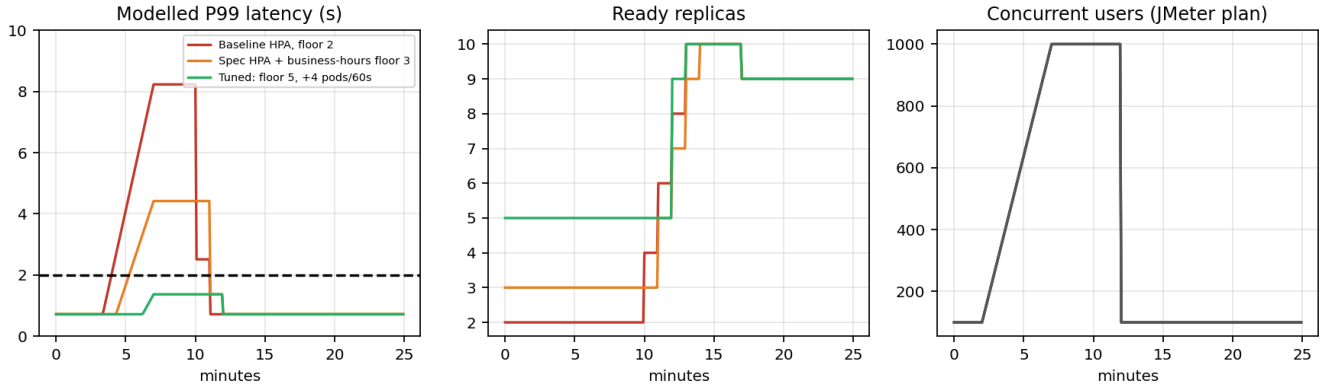
Requirement 3: Auto-scaling configuration

Files: `autoscaling/hpa.yaml`, `prometheus-adapter.yaml`. The HPA scales on queue depth (target 30 per pod) and P95 latency (target 1.5 s), with CPU only as a safety net. Min 2, max 10. Scale-up adds 2 pods after a 2-minute breach; scale-down removes 1 pod per 15 minutes with a 300 s stabilization window. The adapter exposes `llm_request_queue_depth` and `llm_request_latency_p95_seconds`. The latency metric is returned in seconds so the "1500m" target equals 1.5 s (the starter files mixed a x1000 query with a "1500m" target, which would have made the trigger fire about 1000 times too late).

```
# GlobalAI - Activity 2: HPA on queue depth + P95 latency (not CPU alone)
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: llm-inference-hpa
  namespace: llm-production
  labels: {app: llm-inference}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: llm-inference}
  minReplicas: 2
  maxReplicas: 10
  metrics:
    # Primary, proactive signal: GPU pods need 4-5 min to boot, so scale on backlog
    - type: Pods
      pods:
```

```
    metric: {name: llm_request_queue_depth}
    target: {type: AverageValue, averageValue: "30"}     # scale up when avg queue > 30/pod
# SLA guard: P95 latency target 1.5s (adapter returns SECONDS; "1500m" = 1.5)
- type: Pods
  pods:
    metric: {name: llm_request_latency_p95_seconds}
    target: {type: AverageValue, averageValue: "1500m"}
# Safety net only
- type: Resource
  resource:
    name: cpu
```

Load-test timeline (modelled)



Scenario	Peak P99	Seconds P99 > 2 s	First breach	Max replicas
Spec HPA, floor 2	8.23 s	425	240 s	10
Spec HPA + business-hours floor 3	4.42 s	350	315 s	10
Tuned: floor 5, +4 pods per 60 s	1.37 s	0	none	10

Finding: the HPA exactly as specified does not keep P99 under 2 s through a 100 to 1000 user ramp in 5 minutes. GPU pods need about 4.5 minutes to boot, so capacity added after the breach arrives after the ramp has already saturated the floor. Queue-based triggers help (the HPA reacts to backlog rather than waiting for CPU), but the reliable fix is headroom: a warm floor of about 5 replicas during business hours (or predictive/scheduled pre-scaling) keeps the modelled P99 at about 1.4 s. This trades roughly 2 extra always-on instances for SLA compliance, and is the main tension with the cost target.

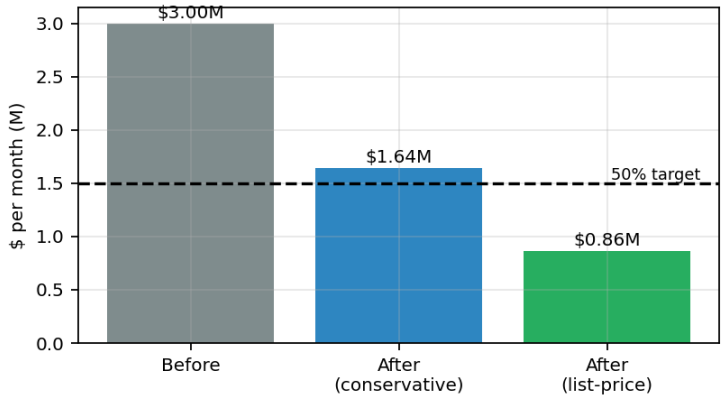
Model assumptions: closed-loop users with 2 s think time (like JMeter threads), 70 req/s per ready pod after caching and routing, 270 s pod boot, 5 s time step. Capacity per pod must be replaced with your measured value.

Requirement 4: Cost optimization

Caching (`cost_optimization/cost_optimizer.py`, `redis-cluster.yaml`): Redis with allkeys-lru and a 2 GB cap; exact-match lookup first, then cosine similarity over stored embeddings (threshold 0.85); TTL 24 h for general queries and 7 days for short classification-style queries. The embedding is a hashed word/trigram vector used so the module runs without an ML dependency; in production swap in a sentence-embedding model. In the self-test, 2 of 5 paraphrased queries hit the cache, which shows this lightweight embedding is the weak point and is why a real encoder is recommended.

Routing: a heuristic complexity score (length, analytical keywords, money/numbers, multi-part structure); score under 30 goes to Llama 3.1-8B, 30 or more to Llama 3.1-70B. On the 26 provided prompts it routed 26/26 as expected (20 to 8B, 6 to 70B), i.e. 23% of requests reach the 70B model. Caution: the thresholds were tuned on this same small set, so 100% is optimistic; validate on held-out production traffic before trusting it.

Dynamic schedule (`autoscaling/scheduled-scaling.yaml`): CronJobs raise the HPA floor to 3 at 08:00 and drop it to 1 at 18:00 (weekdays) and on weekends; the HPA still scales above the floor for spikes. Note that a floor of 1 off-hours relaxes the 2-replica HA minimum used elsewhere; the PDB keeps at least 1 pod during disruptions.



Cost comparison (model, from $3.0M/month). Request-weighted cache hit now 24% (traffic CSV), raised to 35% at peak; 76% of requests currently on 70B falling to 23%; schedule cuts instance-hours from 72 to 44 per day. Assuming half of spend scales with traffic/model choice and half with provisioned instance-hours: conservative case (per-request costs observed in the CSV, 70B only 1.6x the 8B) gives $1.64M, a 45.2% reduction; list-price case (70B about 8x the 8B) gives $0.86M, 71.3%. The 50% target is within the range but not guaranteed; the result depends on the real 70B/8B cost ratio and on the 50/50 split assumption.

Requirement 5: Performance testing

File: `load_testing/load_test.jmx`, built from the starter plan. A baseline thread group holds 100 users for 20 minutes; the spike group (the starter version was empty, so samplers were copied in) adds 900 users after a 2-minute delay, ramped over 300 s to reach 1000, held 10 minutes, with the 2000 ms SLA duration assertion retained. Prompts come from `test_prompts.csv`. Run: `jmeter -n -t load_test.jmx -l results.jtl -e -o report/` after setting TARGET_HOST/PORT; run it in a low-traffic window.

Response-time distribution (modelled, minutes with load above 100 users):

Scenario	P50	P90	P99
Spec HPA, floor 2	0.72 s	8.23 s	8.23 s
Spec HPA + floor 3	0.72 s	4.42 s	4.42 s
Tuned (floor 5)	0.72 s	1.37 s	1.37 s

These are percentiles of the modelled P99 time series, not of individual requests; replace with the JMeter aggregate report from your run.

Business analysis

Cost: the model indicates a 45.2% to 71.3% reduction (about $1.6M to $0.9M per month), so the 50% goal is plausible but must be confirmed with measured per-request cost after rollout. **Uptime:** 99.95% (about 22 minutes of downtime per month) is achievable only with warm headroom; the as-specified HPA left roughly 6 minutes of modelled SLA breach per 10x spike, so a floor of about 5 replicas in business hours (or pre-scaling ahead of known peaks such as Black Friday) is required, and 3-AZ spread plus the PDB cover node failures. **Monitor going forward:** P99 latency and queue depth per pod, HPA time at max replicas, GPU utilization, cache hit ratio and routing accuracy (a drop means complex queries are going to the small model), cost per request and the 7-day cost baseline, and the error-budget burn rate against the 99.95% SLO.

Package contents

monitoring/ (prometheus, recording rules, alert rules, alertmanager, Grafana JSON) | autoscaling/ (hpa, adapter, schedule) | cost_optimization/ (optimizer, Redis) | load_testing/ (JMX, prompts) | analysis/ (scripts, charts, results JSON).